



Informe Técnico de Integración Backend

Requisitos críticos para la defensa del proyecto

De:	Lead Frontend — Equipo Civis
Para:	Equipo Backend
Fecha:	4 de mayo de 2026
Prioridad:	⚠ ALTA — Defensa en 2 semanas

Resumen Ejecutivo

El frontend de Civis está completamente terminado: CSS responsivo, suite de tests con Vitest al 90% de cobertura y lógica de integración preparada. Los siguientes 4 problemas bloquean la experiencia de usuario y deben resolverse antes de la defensa.

#	Problema	Impacto	Endpoint afectado
1	Datos de perfil no persisten tras el registro	BLOQUEANTE	GET /profile
2	Estado de likes se pierde al recargar	ALTO	POST/GET /favorites
3	Username no inyectado en la sesión	ALTO	POST /login
4	Calendario no renderiza (quedado en 'Cargando...')	MEDIO	GET /tramites

BUG #1 — Perfil: datos no persisten tras el registro

Descripción del problema

El formulario de registro envía todos los campos al backend. Tras el login, la vista de perfil llama a GET /profile pero los campos opcionales (dirección, teléfono, fecha de nacimiento, etc.) llegan vacíos o no llegan.

Evidencia visual

Los campos Nombre, Apellido, DNI, Teléfono, Email, Fecha de Nacimiento, Dirección, Localidad, Código Postal y Provincia aparecen vacíos después del login pese a haberse introducido en el registro.

Contrato de API esperado por el frontend

POST /auth/register — Request body

```
{
  "username":    "pedro.perez",
  "email":       "pedro@civis.com",
  "password":    "...",
  "nombre":      "Pedro",
  "apellido":    "Pérez",
  "dni":         "12345678A",
  "telefono":    "666777888",
  "fechaNacimiento": "1990-05-15",
  "direccion":   "Calle Principal 123",
  "localidad":   "Madrid",
  "codigoPostal": "28001",
  "provincia":   "Madrid"
}
```

GET /profile — Response esperada

```
{
  "id":          1,
  "username":    "pedro.perez",
  "email":       "pedro@civis.com",
  "nombre":      "Pedro",
  "apellido":    "Pérez",
  "dni":         "12345678A",
  "telefono":    "666777888",
  "fechaNacimiento": "1990-05-15",
  "direccion":   "Calle Principal 123",
  "localidad":   "Madrid",
  "codigoPostal": "28001",
  "provincia":   "Madrid",
  "rol":         "usuario"
}
```

PUT /profile — Request body (edición desde la vista de perfil)

Mismo esquema que el registro. Todos los campos son opcionales excepto email.

Authorization: Bearer <token> (header en todas las llamadas autenticadas)

El frontend mapea la respuesta directamente a los inputs del formulario.

Si un campo llega null o undefined, el input queda vacío (no rompe, pero es UX pobre).

Checklist de verificación para el backend

- El modelo de usuario en BD incluye todos los campos del contrato
- El serializer/DTO devuelve TODOS los campos (no solo id, email, rol)
- El endpoint GET /profile usa el ID del token JWT, no un parámetro de URL
- Los campos opcionales se guardan como NULL si no se envían (no se ignoran)

BUG #2 — Estado de likes volátil

Descripción del problema

Al pulsar el corazón en un vídeo, la UI responde visualmente de forma inmediata (optimistic update). Al recargar la página el estado se pierde, lo que indica que el backend no está persistiendo la relación usuario↔contenido.

Flujo esperado completo

1. Usuario pulsa corazón → frontend llama POST /favorites
2. Backend guarda relación { userId, tramiteId } en tabla de favoritos
3. Al recargar, GET /tramites devuelve cada trámite con isFavorite: true/false según el usuario autenticado
4. Alternativamente, GET /favorites devuelve lista de IDs favoritos del usuario

Contratos de API

POST /favorites

```
Request: { "tramiteId": 42 }
Response: { "ok": true, "isFavorite": true }
```

DELETE /favorites/:tramiteId

```
Response: { "ok": true, "isFavorite": false }
```

GET /favorites (opcional pero recomendado)

```
Response: { "favoriteIds": [42, 17, 8] }
```

GET /tramites (con favoritos inyectados)

```
{
  "tramites": [
    {
      "id": 42,
      "titulo": "Alta Seguridad Social",
      "isFavorite": true,    // <- el frontend espera este campo
      ...
    }
  ]
}
```

La tabla de favoritos debe tener una restricción UNIQUE (userId, tramiteId).

Si el usuario no está autenticado, isFavorite debe llegar como false (no null).

El frontend NUNCA envía el userId en el body — lo extrae del JWT en el servidor.

Checklist de verificación

- Tabla/colección de favoritos con FK a usuarios y contenidos
- El endpoint POST /favoritos es idempotente (doble click no da error 500)
- GET /tramites inyecta isFavorite basado en el usuario del token
- Sin token: isFavorite = false en todos los trámites

BUG #4 — Calendario atascado en 'Cargando...'

Descripción del problema

La vista del calendario llama a GET /tramites para obtener los trámites con fecha límite y pintarlos en el calendario. Si el endpoint no responde, responde con error o no incluye el campo fechaLimite, el calendario se queda en estado de carga indefinido.

Campo requerido en GET /tramites

El calendario solo necesita estos campos por trámite:

```
{
  "id": 42,
  "titulo": "Alta Seguridad Social",
  "fechaLimite": "2025-06-30", // formato ISO 8601 - OBLIGATORIO
  "categoria": "Seguridad Social",
  "isFavorite": true
}
```

El frontend usa new Date(fechaLimite) directamente.

Formato aceptado: 'YYYY-MM-DD' o ISO 8601 completo '2025-06-30T00:00:00Z'.

Formato NO aceptado: '30/06/2025', '30-06-2025', timestamps Unix.

Si fechaLimite es null, el trámite no se pinta en el calendario (no da error).

Causa probable del bloqueo

- El endpoint devuelve HTTP 401/403 (falta token) y el frontend queda esperando
- El campo se llama diferente en la respuesta (ej. fecha_limite vs fechaLimite)
- La respuesta no es un array sino un objeto sin la key tramites

Formato de respuesta esperado

```
// Opción A — array directo (más simple)
[ { id, titulo, fechaLimite, ... }, ... ]
```

```
// Opción B — objeto con meta (recomendado)
{ "tramites": [...], "total": 12, "page": 1 }
```

Si el backend usa la Opción B, comunicarlo al frontend para ajustar el destructuring.

Checklist de verificación

- GET /tramites responde 200 sin token (los trámites son públicos)
- El campo se llama exactamente fechaLimite (camelCase)
- El formato de fecha es YYYY-MM-DD o ISO 8601

- El endpoint no requiere paginación obligatoria para el primer request

Configuración CORS requerida

Todos los endpoints deben responder con las siguientes cabeceras para que el frontend pueda consumirlos desde el navegador:

Cabecera	Valor requerido
Access-Control-Allow-Origin	http://localhost:5500 (dev) / dominio final
Access-Control-Allow-Methods	GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers	Content-Type, Authorization
Access-Control-Allow-Credentials	true (si se usan cookies de sesión)

El navegador envía una petición OPTIONS antes de POST/PUT con Authorization.

El servidor debe responder 200 a OPTIONS con las cabeceras anteriores.

Sin esto, el login y el guardado de perfil fallan con error CORS en consola.

Estrategia de contingencia: Mock Data

En caso de que el backend no esté completamente listo para la defensa, el frontend puede activar el modo mock con una sola variable. Esto garantiza una demo funcional ante el tribunal.

js/api.js — Flag de control

```
// Cambiar a false cuando el backend esté listo
const USE MOCK = true;

export async function getTramites() {
  if (USE MOCK) {
    const { MOCK_TRAMITES } = await import('./mock-data.js');
    return MOCK_TRAMITES;
  }
  const res = await fetch(`${API_BASE}/tramites`);
  return res.json();
}
```

Tener preparadas dos pestañas del navegador:

- Una con USE MOCK = true (demo garantizada sin depender del servidor)
- Una con USE MOCK = false (demo real si el backend está activo)

El tribunal valorará la preparación, no la suerte de la red.

Tabla resumen de endpoints

Método	Endpoint	Auth	Descripción
POST	/auth/register	No	Registro con todos los campos de perfil
POST	/auth/login	No	Devuelve { token, user }
GET	/profile	Bearer JWT	Datos completos del usuario autenticado
PUT	/profile	Bearer JWT	Actualiza datos de perfil
GET	/tramites	No (público)	Lista con fechaLimite e isFavorite
POST	/favorites	Bearer JWT	Añade trámite a favoritos
DELETE	/favorites/:id	Bearer JWT	Elimina trámite de favoritos
GET	/favorites	Bearer JWT	IDs de favoritos del usuario

Revisión Técnica — Estado de la Integración Frontend

El análisis es rápido porque el código está bien estructurado: los problemas son concretos y pocos. A continuación se detalla qué funciona correctamente y qué inconsistencias deben corregirse antes de la defensa.

Lo que está bien

- **fetchAPI** inyecta el token automáticamente, maneja 401 y errores de conexión. Sólido.
- El registro hace un merge inteligente: `{ ...userData, ...response.user }` — los datos locales se combinan con los del servidor. Buena decisión.
- Login y register ya guardan `response.user` si viene. El Bug #3 no es del frontend — es del backend, que no lo manda.

Inconsistencias reales que hay que corregir

1. El calendario llama al endpoint equivocado

El documento especifica `GET /tramites` para el calendario, pero el código llama a `/calendar`. Son dos cosas distintas y hay que aclararlo con el backend.

- `/calendar` — ¿devuelve eventos genéricos? ¿requiere auth?
- `/tramites` — la videoteca, con `fechaLimite` e `isFavorite`

El calendario del frontend probablemente necesita ambos, o solo `/favorites/calendar` si solo pinta los favoritos del usuario. Necesita confirmarse con el backend.

2. Los favoritos están hardcoded fuera del CONFIG

```
// ❌ Así está ahora - rutas manuales, fuera del sistema centralizado
async function getFavoritesApi() {
  return await fetchAPI('/favorites');
}
async function toggleFavoriteApi(videoId) {
  return await fetchAPI(`/favorites/${videoId}/toggle`, { method: 'POST' });
}

// ✅ Así debería estar - añadir en config.js
favorites:      "/favorites",
toggleFavorite: "/favorites/:id/toggle",
checkFavorite:  "/favorites/:id/check",
favoritesCalendar: "/favorites/calendar",
favoritesUpcoming: "/favorites/upcoming",

// Y en api.js usar CONFIG en vez de rutas literales:
async function toggleFavoriteApi(videoId) {
  return await fetchAPI(
    CONFIG.api.endpoints.toggleFavorite.replace(':id', videoId),
    { method: 'POST' }
  );
}
```

3. getUserProfile tiene una duda que hay que resolver

```
async function getUserProfile() {
  const response = await fetchAPI(CONFIG.api.endpoints.userProfile);
  return response.user || response; // ← este OR es una señal de
  incertidumbre
}
```

El `response.user || response` significa “no sé si el backend envuelve el objeto o no”. El documento especifica que debe devolver el objeto directamente (no envuelto en `{ user: {} }`), así que cuando el backend lo implemente quedará simplemente `return response`. Por ahora el código es defensivo y no rompe en ningún caso — se puede dejar así hasta confirmarlo.

4. El endpoint de perfil está en `/auth/user/profile`

El documento recoge `GET /profile` pero la ruta real implementada es `GET /auth/user/profile`. No es un error crítico, pero hay que mandar al backend la ruta correcta y actualizar la tabla resumen de endpoints si se les vuelve a pasar el informe.

En resumen

Área	Estado
CSS y responsividad	✓ Cerrado
Tests Vitest	✓ 90% cobertura
api.js — lógica de auth	✓ Correcto
api.js — favoritos hardcodeados	⚠ Mover a CONFIG (5 min)
Endpoint calendario vs tramites	⚠ Aclarar con backend
Documento backend — ruta perfil	⚠ Corregir <code>/profile</code> → <code>/auth/user/profile</code>
Backend (bugs #1 al #4)	✗ Pendiente de ellos